

```
appendFile :: FilePath -> String -> IO ()
appendFile f txt = withFile f AppendMode (\hdl ->
hPutStr hdl txt)
```

The `System.Environment` module has useful functions to read command line arguments. The `getArgs` function returns an array of arguments passed to the program. The `getProgName` provides the name of the program being executed. Their type signatures are shown below:

```
ghci> :m System.Environment
```

```
ghci> :t getArgs
getArgs :: IO [String]
```

```
ghci> :t getProgName
getProgName :: IO String
```

Here is an example:

```
import System.Environment
main = do
  args <- getArgs
  program <- getProgName
  putStrLn ( "Program : " ++ program)
  putStrLn "The arguments passed are: "
  mapM putStrLn args
```

Executing the above listed code produces the following output:

```
$ ghc--make args.hs
```

```
[1 of 1] Compiling Main ( args.hs, args.o )
Linking args ...
```

```
$ ./args 1 2 3 4 5
```

```
Program : foo
The arguments passed are:
1
2
3
4
5
```

The `mapM` function is the `map` function that works for `Monad`. Its type signature is:

```
ghci> :t mapM
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
```

The `System.Directory` module has functions to operate on files and directories. A few examples are shown below:

```
ghci> :t createDirectory
createDirectory :: FilePath -> IO ()
```

```
ghci> createDirectory "/tmp/foo"
ghci>
```

If you try to create a directory that already exists, it will return an exception:

```
ghci> createDirectory "/tmp/bar"
*** Exception : /tmp/bar : createDirectory : already
exists (File exists)
```

You can use the `createDirectoryIfMissing` function, and pass a Boolean option to indicate whether to create the directory or not. Its type signature is as follows:

```
ghci> :t createDirectoryIfMissing
createDirectoryIfMissing :: Bool -> FilePath ->
IO ()
```

If `True` is passed and the directory does not exist, the function will create parent directories as well. If the option is `False`, it will throw up an error:

```
ghci> createDirectoryIfMissing False "/tmp/a/b/c"
*** Exception : /tmp/a/b/c : createDirectory : does
not exist (No such file or directory)
```

```
ghci> createDirectoryIfMissing True "/tmp/a/b/c"
```

```
ghci>
```

You can remove directories using the `removeDirectory` or `removeDirectoryRecursive` functions. Their type signatures are as follows:

```
ghci> :t removeDirectory
removeDirectory :: FilePath -> IO ()
```

```
ghci> :t removeDirectoryRecursive
removeDirectoryRecursive :: FilePath -> IO ()
```

A few examples are shown below:

```
ghci> createDirectoryIfMissing True "/tmp/a/b/c"
ghci>
```

```
ghci> removeDirectory "/tmp/a"
*** Exception : /tmp/a : removeDirectory : unsatisfied
constraints (Directory not empty)
ghci> removeDirectoryRecursive "/tmp/a"
```

```
ghci>
```